

Deliverable 5: Service-Oriented Software Engineering Application

Bank Management System

May 16, 2026

1. Introduction

The project is a Bank Management System for retail banking workflows. Before this deliverable, the implemented system was a JavaFX desktop application backed by application services, domain workflows, DAOs, and Microsoft SQL Server.

The goal of Deliverable 5 is to extend the legacy application into a Service-Oriented Software Engineering application. The work follows the required lifecycle:

Identify -> Define -> Design -> Implement -> Deploy -> Validate

The REST API was added as a new adapter layer beside JavaFX. The desktop application remains available through `com.bms.view.BankManagementSystemApp`, while the API runs through the new entry point `com.bms.api.BankManagementApiApplication`.

2. Current Legacy Architecture

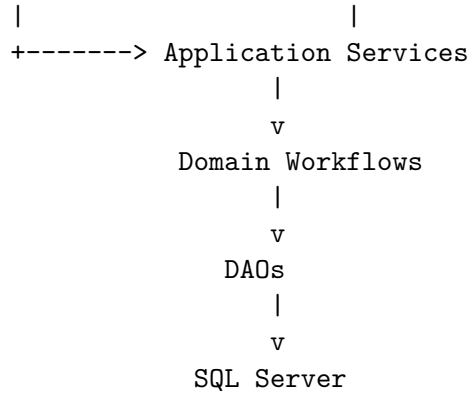
The legacy application has the following structure:

```
JavaFX presentation
|
v
Application services
|
v
Domain workflows and controllers
|
v
DAOs and persistence provider
|
v
Microsoft SQL Server
```

The system was not originally a REST service. There was no servlet, Spring Boot, JAX-RS, embedded HTTP server, or API adapter. Therefore the REST API was added as a separate adapter, not as a replacement for the JavaFX UI.

Updated architecture:

```
JavaFX adapter          REST API adapter
```



3. Service Engineering Process

3.1 Service Candidate Identification

Service	Classification	Main Entity or Workflow
Account Service	Business service	Bank account
Loan Service	Business service	Loan application and review
Customer Onboarding Workflow Service	Coordination service	Customer, account, optional loan

3.2 Justification

The Account Service is reusable by customers, admins, future web apps, mobile apps, and internal banking workflows. It is centered on the account entity and supports querying, status changes, account closure, and account opening.

The Loan Service is reusable by customers and admins. Customers submit and view loan applications, while admins approve, reject, or cancel pending applications. It is independent enough to be consumed by both the JavaFX UI and REST clients.

The Customer Onboarding Workflow Service coordinates multiple services. It creates a customer, opens an account, and optionally submits a starter loan application. This makes it a valid composite service.

3.3 Service Classification

Service	Type	Reason
Account Service	Business	Owns account-facing banking capabilities
Loan Service	Business	Owns loan application lifecycle capabilities
Customer Onboarding Workflow Service	Coordination	Orchestrates account and loan services

4. Service Requirements Specification

Functional Requirements

Requirement	Description
FR-1	Retrieve account details by account number
FR-2	Retrieve loans for a customer
FR-3	Create a customer profile and initial account
FR-4	Submit a loan application
FR-5	Update account status
FR-6	Approve or reject a pending loan
FR-7	Close an account using logical status update
FR-8	Cancel a pending loan
FR-9	Execute customer onboarding as a composite workflow

Non-Functional Requirements

Requirement	Description
Reusability	Services are called through application service classes rather than UI code
Separation of concerns	API DTOs are separate from domain entities
Maintainability	REST controllers do not call DAOs directly
Security hygiene	API responses do not serialize passwords
Monetary correctness	New API contracts use <code>BigDecimal</code> for money-facing inputs
Interoperability	API contract is documented in OpenAPI YAML

Inputs, Outputs, and Constraints

Inputs are JSON request bodies and path parameters. Outputs are JSON response DTOs. Constraints include SQL Server as the implemented database target, coursework-level authentication, and logical deletion for critical financial data.

5. REST API Design

The REST resources are:

```
Customer
|
+-- Account
|
+-- Loan
```

Endpoint design:

Method	Endpoint	Purpose
GET	/api/accounts/{accountNumber}	Get account details
GET	/api/customers/{customerId}/loans	Get customer loans
POST	/api/customers	Create customer and initial account
POST	/api/loans	Submit loan application
PUT	/api/accounts/{accountNumber}/status	Update account status
PUT	/api/loans/{loanId}/decision	Approve or reject loan
DELETE	/api/accounts/{accountNumber}	Close account
DELETE	/api/loans/{loanId}	Cancel pending loan
POST	/api/workflows/customer-onboarding	Composite onboarding workflow

Standard response statuses are documented: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, and 500 Internal Server Error.

6. REST API Implementation

Technology Used

The REST adapter uses Spring Boot with Java 17. Maven dependencies were added for:

```
spring-boot-starter-web
spring-boot-starter-validation
```

The API entry point is:

```
com.bms.api.BankManagementApiApplication
```

The original JavaFX entry point is unchanged:

```
com.bms.view.BankManagementSystemApp
```

Package Structure

```
src/main/java/com/bms/api
src/main/java/com/bms/api/config
src/main/java/com/bms/api/controller
src/main/java/com/bms/api/dto
src/main/java/com/bms/api/mapper
```

Service Layer Integration

The API controllers call application services only:

Controller	Application Services
AccountController	AccountQueryService, AccountManagementService
CustomerController	AccountManagementService
LoanController	LoanApplicationService

Controller	Application Services
WorkflowController	CustomerOnboardingWorkflowService

New or adjusted service capabilities:

Class	Change
AccountQueryService	Added account and transaction query facade
AccountManagementService	Added customer plus account creation and close account operation
LoanApplicationService	Added pending loan submission and pending loan cancellation
CustomerOnboardingWorkflowService	Added composite customer onboarding workflow

DAO additions were made below the service layer:

DAO	Change
AccountDAO	Added account insert
CustomerDAO	Added national ID existence check
LoanDAO	Added pending loan cancellation

7. Service Composition

Composite service:

POST /api/workflows/customer-onboarding

Workflow diagram:

```

POST /api/workflows/customer-onboarding
|
v
CustomerOnboardingWorkflowService
|
+--> AccountManagementService.createCustomerProfile(...)
|
+--> AccountManagementService creates initial account
|
+--> LoanApplicationService.submitLoanApplication(...)
|
v

```

Combined workflow response

The orchestration is implemented in the service layer, not inside the REST controller. The controller only maps JSON DTOs to service inputs and maps the result to an HTTP response.

8. OpenAPI Documentation

The OpenAPI contract is stored at:

`docs/deliverable5/openapi.yaml`

It documents all endpoints, path parameters, request schemas, response schemas, and standard HTTP status codes. Password fields are marked as write-only in the API specification and are not present in response schemas.

9. Testing

The Postman collection is stored at:

`docs/deliverable5/postman_collection.json`

Collection requests:

Name	Method and Endpoint
01_GET_account_by_number	GET /api/accounts/{accountNumber}
02_GET_customer_loans	GET /api/customers/{customerId}/loans
03_POST_create_customer	POST /api/customers
04_POST_submit_loan	POST /api/loans
05_PUT_update_account_status	PUT /api/accounts/{accountNumber}/status
06_PUT_decide_loan	PUT /api/loans/{loanId}/decision
07_DELETE_close_account	DELETE /api/accounts/{accountNumber}
08_DELETE_cancel_loan	DELETE /api/loans/{loanId}
09_POST_customer_onboarding_workflow	POST /api/workflows/customer-onboarding

Required screenshots for final submission:

Screenshot	Must Show
GET request	URL, method, status code, JSON response
POST request	URL, method, request body, status code, JSON response
PUT request	URL, method, request body, status code, JSON response
DELETE request	URL, method, status code, JSON response
Composite workflow	URL, method, request body, status code, combined JSON response

Sample request:

```
{  
  "customerId": 1,  
  "amount": 50000.00,  
  "termMonths": 24,
```

```
"loanPurpose": "Education",  
"loanType": "EDUCATION"  
}
```

Sample response:

```
{  
  "loanId": 15,  
  "status": "PENDING",  
  "message": "Loan application submitted successfully"  
}
```

10. Assumptions and Limitations

The API uses coursework-level actor fields such as `actorId`, `adminId`, and `role`. It does not implement production token authentication.

`AuthContext` is not used for REST session handling because it is a desktop singleton and is not request-safe for HTTP APIs.

Passwords are still stored by the legacy database model as plaintext, but API responses do not expose password fields. Production usage would require password hashing.

Money-facing API DTOs use `BigDecimal`, even though some legacy loan entity fields still use `double`.

Account closure uses logical deletion by setting account status to `CLOSED`.

Loan cancellation uses logical state update to `CANCELLED`; the SQL schema was updated to allow this status. For existing local databases created before this schema update, the implementation falls back to deleting only pending loan applications if the legacy status check constraint rejects `CANCELLED`.

SQL Server is the only concrete persistence backend implemented.

No cloud deployment is required for this coursework deliverable. Deployment is local through the Spring Boot runtime on port 8080.

Multi-step workflows are not fully transaction-scoped across all DAO calls. For production-grade banking, customer/account creation and financial workflows should use explicit transaction boundaries.

11. Conclusion

Deliverable 5 extends the legacy JavaFX Bank Management System with a REST API adapter and service-oriented structure. The implementation identifies reusable Account and Loan business services, adds a Customer Onboarding coordination service, documents all endpoints in OpenAPI, and provides a Postman collection for validation.

The architecture now supports both desktop and REST clients through the application service layer:

```
Client / Postman / Future Web App
|
v
REST API Controllers
|
v
Application Services
|
v
Domain Workflows
|
v
DAOs / Persistence Provider
|
v
SQL Server
```